JavaScript Course

Comprehensive Notes

Emmanuel Adedayo

Code Academy | SEA Technologies Media

Learn. Build. Innovate.

SEA TECHNOLOGIES MEDIA

Topic 1: What is JavaScript and How It Works

JavaScript is a high-level, dynamic programming language that brings interactivity and behavior to websites. While HTML provides structure and CSS adds style, JavaScript is what makes web pages come alive with dynamic content, animations, and user interaction.

Key Concepts

- **High-level language** — JavaScript abstracts low-level details like memory management, making it easier to write and read.
- **Interpreted language** — JavaScript code is executed line by line by the browser's JavaScript engine (e.g., V8 in Chrome, SpiderMonkey in Firefox).
- **Dynamic typing** — Variables can hold any type of value, and types can change at runtime.
- **Client-side and server-side** — JS runs in the browser (client-side) and on servers via Node.js (server-side).
- **Event-driven** — JavaScript responds to user actions like clicks, key presses, and form submissions.
- **JavaScript is NOT Java** — Despite the similar name, they are entirely different languages with different purposes.

JavaScript can be added to a webpage in three ways:

- **Inline JavaScript** — Written directly inside HTML attributes (e.g., `onclick` in a button tag). Quick for simple actions but not recommended for large projects.
- **Internal JavaScript** — Placed inside `<script>` tags within the HTML file. Good for small scripts specific to one page.
- **External JavaScript** — Linked via `<script src="file.js">` in the HTML. Best practice for maintainability and reusability.

The browser loads HTML first, then encounters `<script>` tags and executes the JavaScript. External scripts are cached, improving load times on repeat visits.

Tip

Understanding the Role of JavaScript
Think of a webpage as a person: HTML is the skeleton, CSS is the skin and clothing, and JavaScript is the brain and muscles. Together, they create a complete, interactive experience.

Warning

Common Misconception
New learners often confuse JavaScript with Java. Remember: JavaScript runs in web browsers (and servers via Node.js) to add interactivity, while Java is a general-purpose language used for enterprise applications, Android apps, and more. They share almost nothing beyond the name.

Key takeaway: JavaScript is the programming language of the web. It adds behavior and interactivity to websites and can run both in the browser and on the server. Learning it is

> essential for modern web development.

Topic 2: Variables and Data Types

Variables are containers for storing data values. JavaScript provides three ways to declare variables: `const`, `let`, and `var`. Understanding the differences between them is crucial for writing reliable code.

Key Concepts

- **`const`** — Declares a variable whose value cannot be reassigned. Use this by default for values that should not change.
- **`let`** — Declares a variable whose value can be reassigned. Use this when the value needs to change.
- **`var`** — The old way to declare variables (pre-ES6). Has function scope and is hoisted. Avoid using `var` in modern code.
- **String** — Text data enclosed in quotes: `"Hello"`, `'World'`, or template literals `` `Hello ${name}` ``.
- **Number** — Integer and floating-point values: `42`, `3.14`, `-7`.
- **Boolean** — Logical values: `true` or `false`.
- **null** — Intentional absence of any value. You assign it explicitly.
- **undefined** — A variable declared but not assigned a value.
- **typeof** operator — Returns the type of a variable as a string.

Variable naming rules: names can contain letters, digits, underscores, and dollar signs, but must not start with a digit. Use **camelCase** for variable names in JavaScript (e.g., `firstName`, `isLoggedIn`).

Tip

Always Use `const` by Default
Start every variable declaration with `const`. Only switch to `let` if you genuinely need to reassign the value later. This prevents accidental reassignment and makes your code more predictable and easier to debug.

Warning

`var` Is Outdated
The `var` keyword has function scope instead of block scope, which can cause unexpected behavior. It is also hoisted to the top of its scope, meaning it can be used before declaration. Always prefer `const` and `let` over `var`.

Key takeaway: Use `const` for values that never change and `let` for values that do. Avoid `var`. JavaScript has seven primitive data types: string, number, boolean, null, undefined, symbol, and bigint.

Topic 3: Operators

Operators are symbols that perform operations on values and variables. JavaScript supports arithmetic, comparison, logical, and assignment operators among others. Understanding them is fundamental to writing expressions and logic.

Key Concepts

- **Arithmetic operators** — `+`, `-`, `*`, `/`, `%` (modulo), `**` (exponentiation).
- **Assignment operators** — `=`, `+=`, `-=`, `*=`, `/=` (shorthand for combining assignment with arithmetic).
- **Comparison operators** — `===` (strict equal), `!==` (strict not equal), `==` (loose equal), `!=` (loose not equal), `>`, `<`, `>=`, `<=`.
- **Logical operators** — `&&` (AND), `||` (OR), `!` (NOT).
- **String concatenation** — The `+` operator joins strings: `"Hello" + " World"` produces `"Hello World"`.
- **Strict vs. Loose equality** — `===` checks both value and type; `==` only checks value (with type coercion). Always prefer `===`.

Operator precedence determines the order in which operators are evaluated. Multiplication and division happen before addition and subtraction, just like in math. Use parentheses to override the default order.

Tip

Always Use `===` Instead of `==`
The loose equality operator (`==`) performs type coercion, so `5 == "5"` returns `true`. The strict equality operator (`===`) compares both value and type, so `5 === "5"` returns `false`. Strict comparison prevents surprising bugs.

Warning

Type Coercion Pitfalls
JavaScript automatically converts types in unexpected ways with loose equality: `null == undefined` is `true`, `"0" == false` is `true`, but `null === undefined` is `false`. Relying on coercion leads to bugs that are hard to trace. Always use strict comparison.

Key takeaway: Operators let you compute, compare, and combine values. Always use strict equality (`===`) over loose equality (`==`), and understand the difference between arithmetic, comparison, and logical operators.

Topic 4: Conditions and Loops

Conditional statements let your program make decisions. Loops let your program repeat actions. Together, they form the backbone of program logic and control flow.

Key Concepts

- **if / else if / else** — Executes code based on conditions. The first true condition runs its block; `else` catches all remaining cases.
- **switch** — Compares a value against multiple cases. Use when you have many discrete options (e.g., days of the week, menu choices).
- **Ternary operator** — Shorthand for simple if/else: `condition ? valueIfTrue : valueIfFalse`.
- **`for` loop** — Repeats a block a known number of times: `for (let i = 0; i < 5; i++)`.
- **`while` loop** — Repeats while a condition is true: `while (condition) { ... }`.
- **`for...of` loop** — Iterates over iterable objects (arrays, strings): `for (const item of array)`.
- **`break`** — Exits a loop early.
- **`continue`** — Skips the current iteration and moves to the next.

Tip

Choose the Right Loop
Use `for` when you know how many times to loop. Use `while` when the number of iterations depends on a condition. Use `for...of` when iterating over arrays — it is cleaner and less error-prone than a traditional `for` loop.

Warning

Infinite Loops
If a loop's condition never becomes false, the loop runs forever, freezing the browser or crashing Node.js. Always ensure your loop has a clear exit condition, especially with `while` loops. Double-check that your counter variable is being updated.

Key takeaway: Conditions control what code runs; loops control how many times code runs. Master `if/else`, `switch`, and all loop types to build logic that makes decisions and repeats efficiently.

Topic 5: Functions

Functions are reusable blocks of code that perform a specific task. They help you organize, reuse, and simplify your code. Instead of writing the same logic multiple times, you define it once in a function and call it whenever needed.

Key Concepts

- **Function declaration** — `function greet() { ... }`. Hoisted, so you can call it before its definition.
- **Function expression** — `const greet = function() { ... }`. Not hoisted; must be defined before use.
- **Arrow function** — `const greet = () => { ... }` or `const add = (a, b) => a + b`. Concise syntax, no own `this`.
- **Parameters** — Variables listed in the function definition (e.g., `function add(a, b)`).
- **Arguments** — The actual values passed when calling the function (e.g., `add(3, 5)`).
- **Return value** — The output a function sends back using `return`. Without `return`, the function returns `undefined`.
- **Default parameters** — `function greet(name = "Guest")` provides a fallback value if no argument is passed.
- **Scope** — Variables defined inside a function are local to that function and not accessible outside.

Tip

Write Small, Focused Functions
Each function should do one thing well. If a function grows too long, split it into smaller helper functions. This makes code easier to read, test, and debug. A good rule of thumb: if you cannot describe what a function does in one sentence, it is doing too much.

Warning

Forgetting `return`
A function without a `return` statement returns `undefined`. If you expect a result but get `undefined`, check that every code path in your function includes a `return` statement. This is one of the most common beginner mistakes.

Key takeaway: Functions make code reusable and organized. Learn the three ways to create them (declaration, expression, arrow), understand parameters and return values, and always write small, focused functions that do one thing well.

Topic 6: Arrays and Objects

Arrays and objects are JavaScript's two fundamental data structures. Arrays store ordered lists of values, while objects store key-value pairs. Together, they let you model virtually any kind of data.

Key Concepts

- **Array** — An ordered collection: `const fruits = ["apple", "banana", "orange"]`. Access items by index (starting at 0).
- **Common array methods** — `.push()`, `.pop()`, `.shift()`, `.unshift()`, `.splice()`, `.indexOf()`, `.includes()`, `.length`.
- **`.forEach()`** — Iterates over each element without creating a new array.
- **`.map()`** — Creates a new array by transforming each element.
- **`.filter()`** — Creates a new array with elements that pass a test.
- **Object** — A collection of key-value pairs: `const student = { name: "Emmanuel", age: 20 }`. Access via dot notation or brackets.
- **`Object.keys()`** — Returns an array of the object's keys.
- **`Object.values()`** — Returns an array of the object's values.
- **`Object.entries()`** — Returns an array of [key, value] pairs.
- **Array of objects** — A common pattern for storing structured data: `const students = [{name: "Ada", grade: "A"}, {name: "Bob", grade: "B"}]`.

Tip

Use `.map()` and `.filter()` Over Loops
Instead of using `for` loops to transform or filter arrays, use `.map()` and `.filter()`. They are more readable, less error-prone, and follow the functional programming style. For example, `students.filter(s => s.grade === "A")` is clearer than a manual loop.

Warning

`const` Arrays and Objects Can Be Modified
Declaring an array or object with `const` prevents reassignment (you cannot replace the whole array), but you CAN still modify its contents (add, remove, or change items). `const` protects the reference, not the contents. If you need truly immutable data, use `Object.freeze()`.

Key takeaway: Arrays hold ordered lists; objects hold key-value pairs. Master array methods (`map`, `filter`, `forEach`) and object access patterns. These are the building blocks for all real-world JavaScript data.

Topic 7: DOM Manipulation

The Document Object Model (DOM) is a programming interface that represents the structure of an HTML document as a tree of objects. JavaScript uses the DOM to read, modify, add, and remove HTML elements dynamically — making pages interactive without reloading.

Key Concepts

- **DOM** — A tree-like representation of the HTML document. Every HTML element is a node in this tree.
- **`document.getElementById()`** — Selects an element by its `id` attribute.
- **`document.querySelector()`** — Selects the first element matching a CSS selector.
- **`document.querySelectorAll()`** — Selects all elements matching a CSS selector (returns a NodeList).
- **`.textContent`** — Gets or sets the plain text inside an element.
- **`.innerHTML`** — Gets or sets the HTML content inside an element (can insert tags).
- **`.style`** — Accesses and modifies inline CSS styles.
- **`.classList`** — Provides methods: `.add()`, `.remove()`, `.toggle()` for CSS classes.
- **`.addEventListener()`** — Attaches an event handler to an element (e.g., `"click"`, `"keydown"`).
- **`.createElement()`** — Creates a new DOM element programmatically.
- **`.appendChild()`** — Adds a child element to a parent node.
- **`.removeChild()`** — Removes a child element from its parent.

Tip

`querySelector` Over `getElementById`
`querySelector` is more flexible — it accepts any CSS selector, not just IDs. You can select by class (`".myclass"`), attribute (`"[data-type=ẍ]"`), or tag (`"h1"`). It is the modern, consistent way to select elements.

Warning

`innerHTML` Security Risk
Never use `innerHTML` with user-supplied data. It can execute malicious scripts (cross-site scripting / XSS). Use `textContent` for plain text and `createElement` + `appendChild` for building elements safely.

Key takeaway: The DOM lets JavaScript interact with HTML. Learn to select elements, modify content and styles, handle events, and dynamically create or remove elements. This is what makes web pages interactive.

★ Topic 8: Final Project — Interactive Student Dashboard

This final project brings together everything you have learned throughout the course. You will build a complete, interactive Student Dashboard using HTML, CSS, and JavaScript.

Project Overview:

The Interactive Student Dashboard is a single-page application that manages student records. It features real-time search, grade filtering, statistics display, and the ability to add and remove students. Every JavaScript concept covered in this course is applied here.

Key Concepts

- **Variables** — `let` for state that changes (student list, count), `const` for DOM references and fixed values.
- **Arrays** — Student records stored as an array of objects: `[{id, name, course, grade}, ...]`.
- **Objects** — Each student is an object with `name`, `course`, `grade`, and `id` properties.
- **Functions** — Modular functions for each action: `updateDisplay()`, `addStudent()`, `removeStudent()`, `getFilteredStudents()`.
- **DOM Manipulation** — Reading form inputs, rendering lists, updating statistics, showing/hiding forms.
- **Events** — Click handlers for buttons, keyboard listener for Enter key, input events for search.
- **Conditions** — Filtering students by grade, validating form input, toggling UI state.
- **Template literals** — Dynamic HTML generation with `` `<div>${student.name}</div>` ``.

Tip

Build Incrementally
Start with the HTML structure and CSS, then add JavaScript one feature at a time: first display the data, then add search, then add/remove functionality. Test after each feature to catch issues early.

Warning

Validate User Input
Before adding a student, check that all fields are filled and contain valid data. Never trust user input — always validate on the client side (and server side in production) to prevent errors and security issues.

Project Features:

1. Student list display with name, course, and grade
2. Real-time search filtering by name
3. Grade filter dropdown (All, A, B, C, D, F)
4. Statistics panel (total students, grade distribution)

5. Add new student form with validation
6. Remove student with confirmation
7. Responsive layout that works on different screen sizes

> **Key takeaway:** This project demonstrates the power of combining variables, arrays, objects, functions, DOM manipulation, events, and conditions into a real application. Build it step by step, and you will see how all the pieces of JavaScript work together.